

IaaS, PaaS, SaaS & AWS EC2 – QUESTIONS & ANSWERS

1. List three popular apps you use (like Instagram, Zomato, or Spotify) and for each, identify whether it is most similar to IaaS, PaaS, or SaaS. Briefly explain your reasoning for each choice.

Answer: The following apps are best understood as **SaaS** from an end-user perspective because users directly access the finished software over the internet without managing servers or the underlying infrastructure.

App	Model	Reason
Instagram	SaaS	Users directly use the completed social-media application through a web browser or mobile app; they do not manage servers.
Zomato	SaaS	Users access the finished food-discovery and ordering service without managing the application's infrastructure.
Spotify	SaaS	Users directly use the music-streaming application to search, play, and manage music without managing the underlying cloud resources.

2. Create a table comparing IaaS, PaaS, and SaaS with at least three points for each model, using real-world examples such as AWS EC2, Google App Engine, and Office 365.

Point	IaaS	PaaS	SaaS
Meaning	Infrastructure as a Service – provides virtual servers, storage, and networking.	Platform as a Service – provides a platform for developing and deploying applications.	Software as a Service – provides ready-to-use software over the internet.
User manages	OS, applications, and configurations; the provider manages physical infrastructure.	Mainly application code and data; the provider manages the platform and infrastructure.	Mostly the application settings and user data; the provider manages the software and infrastructure.
Example	AWS EC2	Google App Engine	Microsoft Office 365
Typical use	Running servers, websites, applications, and custom environments.	Building and deploying applications without managing the underlying servers.	Using ready-made applications such as email, documents, and collaboration tools.

3. Sign up for a free AWS account (or use an existing one), navigate to the EC2 dashboard, and launch a new t2.micro EC2 instance running Ubuntu 22.04. Take a screenshot of your running instance in the AWS console.

Answer – Steps to launch the EC2 instance:

1. Log in to the **AWS Management Console** and open **EC2**.
2. Select the required AWS Region and open the **EC2 Dashboard**.
3. Click **Launch Instance**.
4. Enter an instance name and select an **Ubuntu 22.04 LTS** AMI that is available/eligible for the applicable free-tier offering.
5. Select the required instance type. If **t2.micro** is available in the selected region/account, select it; otherwise use the current free-tier-eligible instance type shown by AWS.
6. Create or select a key pair and download the **.pem** private-key file. Keep it secure.
7. Configure the security group to allow **SSH (port 22)** from your IP address, and allow HTTP (port 80) if the instance will host a website.
8. Click **Launch Instance** and wait until the instance state becomes **Running**.
9. Open the Instances page and take a screenshot showing the running instance.

Screenshot: Attach your own AWS EC2 Instances page screenshot here showing the instance in the **Running** state.

4. After your EC2 instance is running, connect to it using SSH from your local terminal or an online SSH client. Run the command `uname -a` and note the output.

Answer: First, connect to the Ubuntu EC2 instance using SSH and then run:

```
uname -a
```

The command displays system information such as the Linux kernel name, hostname, kernel release/version, machine architecture, and related build information.

Example output format: Linux ip-172-31-xx-xx 5.x.x-xxx-aws #... x86_64 GNU/Linux. The exact output depends on the Ubuntu AMI and kernel version used. For the assignment, paste the actual output from your own EC2 instance.

5. Suppose you want to host a simple static website for your college fest using your EC2 instance. List the basic steps you would follow after launching the instance to make your site accessible to everyone. Write 3–5 steps in order.

Answer – Basic steps:

1. Connect to the Ubuntu EC2 instance using SSH and update the system packages.
2. Install a web server such as **Apache** or **Nginx**.
3. Upload/copy the college-fest HTML, CSS, JavaScript, and image files into the web server's document-root directory.
4. Allow **HTTP port 80** in the EC2 Security Group and ensure the web server is running.
5. Open the instance's public IPv4 address in a browser to access the website.

Important: AWS pricing and free-tier eligibility can change. Always check the current AWS Console pricing/free-tier information before launching resources.